

文章笔记

Karpathy 谈 Coding Agent 与 AI 的未来

五道口纳什整理

2026 年 3 月 29 日

原始来源: Andrej Karpathy on *No Priors* Podcast (Sarah Guo 主持)

译者: 宝玉 @dotey

译文发布日期: 2026 年 3 月 20 日

原始视频: <https://www.youtube.com/watch?v=kwSVtQ7dziU>

译文链接: <https://x.com/dotey/status/2035141635713941927>

本笔记基于宝玉 (@dotey) 的中文译文整理, 原文为 Karpathy 在 No Priors 播客的英文访谈。

目录

1 编码 Agent 时代：“写代码”不再是正确的动词	2
1.1 多 Agent 并行：从 GPU 焦虑到 Token 焦虑	2
1.2 本章小结	2
2 Agent 的灵魂：个性设计与持久化	2
2.1 从单 Agent 到持久化 Claw	2
2.2 为什么个性设计很重要	3
2.3 Dobby 精灵：三个提示词接管一整个家	3
2.4 Agent 优先的互联网	3
2.5 本章小结	4
3 AutoResearch：递归自我改进的试验田	4
3.1 核心理念：把自己作为瓶颈移除	4
3.2 AutoResearch 的实践成果	4
3.3 从 program.md 到元优化	4
3.4 分布式 AutoResearch：互联网 Agent 集群	5
3.5 本章小结	5
4 AI 的参差性：天才 PhD 与 10 岁小孩	5
4.1 可验证领域 vs 不可验证领域	5
4.2 强化学习的优化盲区	6
4.3 一个模型还是一千个大脑	6
4.4 本章小结	6
5 开源与闭源：中心化的历史记录很差	7
5.1 差距在缩小	7
5.2 为什么不回前沿实验室	7
5.3 本章小结	7
6 就业市场与物理世界：原子比比特难一百万倍	8
6.1 数字 AI 的三阶段路线图	8
6.2 就业影响：杰文斯悖论	8
6.3 机器人与信息市场	8
6.4 本章小结	9
7 MicroGPT 与 Agent 时代的教育	9
7.1 200 行代码的极简 GPT	9
7.2 教育范式的转变：向 Agent 解释	9
7.3 本章小结	10
8 总结与延伸	10
8.1 核心论点回顾	10
8.2 三个值得持续关注的信号	10

8.3 拓展阅读	10
----------------	----

1 编码 Agent 时代：“写代码”不再是正确的动词

Andrej Karpathy——OpenAI 联合创始成员、前 Tesla AI 总监、Eureka Labs 创始人——在本次访谈中透露，他从 2024 年 12 月起就基本没有手写过一行代码。他的日常工作模式从“写代码”彻底转变为“指挥 Agent 干活”。

工作模式的翻转

2024 年 12 月是分水岭。Karpathy 在那个时间点从“自己写 80%，Agent 写 20%”翻转到了“自己写 20%，Agent 写 80%”。到访谈时（2026 年 3 月），这个比例可能更加极端。

“写代码已经不是对的动词了。我每天得花 16 个小时指挥我的 Agent 干活。”

——Karpathy

1.1 多 Agent 并行：从 GPU 焦虑到 Token 焦虑

Karpathy 提到 Peter Steinberger（OpenClaw 创始人，后加入 OpenAI）的工作方式：屏幕上铺满 Codex Agent 窗口，同时在十几个代码仓库之间切换，给不同 Agent 分配互不冲突的任务。操作单位不再是“写一行代码”，而是“这个功能交给 Agent 1，那个功能交给 Agent 2”。

Karpathy 用了一个精准的类比：读 PhD 时，GPU 闲着就焦虑——浪费了算力。现在焦虑的对象变成了 Token。

从算力约束到人力约束

过去十年，很多工程任务中人们并不觉得受算力约束。但 AI Agent 的能力跳跃改变了这一点：约束不再是计算资源，而是你自己——你能指挥多少 Token 吞吐量？你能并行管理多少 Agent？

Sarah Guo 补充了一个场景：她在 Conviction（AI 投资基金）的工程团队，所有人都不手写代码，工程师们佩戴麦克风对着 Agent 低声说话。

Karpathy 认为当前做项目的瓶颈是“技能问题”（skill issue）——不是能力不够，而是你还没找到怎么把现有能力串起来。Agent 指令写得不够好、记忆工具不够成熟——出问题时总觉得是自己没搞对。

1.2 本章小结

- 2024 年 12 月是 Karpathy 工作模式的转折点，此后基本不再手写代码
- 多 Agent 并行已成常态，核心度量从 GPU 利用率转向 Token 吞吐量
- 当前瓶颈在于人——如何更好地指挥 Agent，而非 Agent 本身的能力

2 Agent 的灵魂：个性设计与持久化

2.1 从单 Agent 到持久化 Claw

Karpathy 提出了 Claw（爪子）的概念：一种比普通 Agent 更“持久”的实体——有自己的沙箱、成熟的记忆系统，即使你不看着它也在循环运行。他认为 OpenClaw 在记忆系统上远比默认 Agent

工具成熟：默认机制只是在上下文窗口满了之后做一次压缩，而 OpenClaw 有更精细的方案。

2.2 为什么个性设计很重要

Karpathy 对比了几家的 Agent 个性：

- **Claude Code**：感觉像一个队友，会跟你一起兴奋。Anthropic 在“拍马屁的程度”上把握得不错——不成熟的想法只会得到平淡回应，但真正好的想法会获得积极反馈。Karpathy 发现自己在试图“赢得 Claude 的称赞”。
- **OpenAI Codex Agent**：冷冰冰的。ChatGPT 里很活泼，但编程 Agent 版非常干燥——不在乎你在做什么，“噢，我实现了”，没有更多反应。

灵魂文档 (SOUL.md)

Peter Steinberger 在 OpenClaw 上的一个常被忽视但极其重要的创新是“灵魂文档” (SOUL.md)，用来定义 Agent 的性格。Karpathy 认为很多工具低估了个性设计的重要性——Agent 的个性直接影响用户的使用体验和工作效率。

2.3 Dobby 精灵：三个提示词接管一整个家

Karpathy 在 2026 年 1 月建了一个叫 Dobby the Elf Claw 的家庭自动化 Agent。流程极其简洁：

1. 告诉 Agent “我家好像有 Sonos 音箱，你能找到它吗”
2. Agent 扫描局域网、找到设备、逆向工程 API
3. 三个提示词后，音乐就在书房响了

灯光、暖通空调、窗帘、泳池、水疗设备、安防系统——全部用同样的流程接管。安防部分用变化检测 + Qwen 视觉模型分析，通过 WhatsApp 推送消息（如“一辆 FedEx 货车刚停下来，你可能收到了快递”）。

以前需要 6 个不同的 App 控制智能家居，现在一个都不需要——Dobby 用自然语言处理一切。

2.4 Agent 优先的互联网

由 Dobby 引出的更大判断：

行业重构的方向

App Store 里的那些智能家居 App 在某种意义上“不应该存在”。应该只有 API，Agent 直接调用。

核心判断：行业必须在很多方面重新配置。客户不再是人类了，是代表人类行事的 Agent。一两年内，这些能力会变成基本门槛，免费，连开源模型都能做。软件将变成“临时性软件” (ephemeral software)。

2.5 本章小结

- Agent 正在从一次性工具进化为持久化实体 (Claw)，具备记忆、沙箱和自主循环能力
- 个性设计 (灵魂文档) 对 Agent 使用体验的影响被严重低估
- 家庭自动化案例展示了 Agent 统一异构系统的强大能力
- 未来的互联网可能是 Agent 优先的——App 让位于 API，人让位于 Agent 作为客户

3 AutoResearch：递归自我改进的试验田

3.1 核心理念：把自己作为瓶颈移除

Karpathy 的核心原则

“要充分利用已有工具的全部潜力，你必须把自己作为瓶颈移除。”

你不能在那里等着提示下一步。你要安排好一切，让系统完全自主。当下竞争的本质就是增加杠杆——你偶尔投入一点 Token，大量的事情代替你发生。

3.2 AutoResearch 的实践成果

AutoResearch (2026 年 3 月 7 日开源) 的核心是一个约 630 行的 Python 脚本，让 AI Agent 在单 GPU 上自主循环运行实验。Karpathy 用它优化自己的 nanochat (精简 LLM 训练框架) 项目。

nanochat 与递归自我改进

很多人对 Karpathy 痴迷训练 GPT-2 级别的模型感到困惑。但对他来说，这是递归自我改进 (recursive self-improvement) 的试验田——用 AI 来改进训练 AI 的代码——这正是所有前沿实验室都在追求的方向。

Karpathy 已经用“老派”方式 (二十年研究经验 + 大量超参数搜索) 把 nanochat 调得相当好。然后他让 AutoResearch 跑了一个晚上，结果：

- Agent 发现了他没注意到的优化：value embeddings 上的 weight decay 漏了，Adam 优化器的 beta 参数没调够
- 这些参数之间有联合交互效应——调了一个，其他的最优值也变了
- 公开数据：两天约 700 次实验，发现约 20 个叠加有效的改进
- “Time to GPT-2” 指标从 2.02 小时降到 1.80 小时——11% 效率提升
- Shopify CEO Tobias Lütke 用同样方法获得了 19% 性能提升

3.3 从 program.md 到元优化

Sarah Guo 问：什么时候模型能写出比你更好的 program.md?

Karpathy 展开了一个框架：每个研究组织都可以用一组 Markdown 文件来描述——角色、流程、连接方式。不同的 program.md 会产生不同的研究进展。可以想象让多个“研究组织”竞赛，分析改进来源，再用分析结果让模型生成更好的 program.md。

洋葱层模型

LLM 被视为理所当然 → Agent 被视为理所当然 → Claw 实体被视为理所当然 → 可以有多个 → 可以有指令 → 可以优化指令。每一层都无限延伸。

“这就是为什么会到精神错乱的地步——这是无限的，一切都是技能问题。”

3.4 分布式 AutoResearch: 互联网 Agent 集群

Karpathy 正在思考如何将 AutoResearch 并行化，特别是引入互联网上的不可信工作者。他的设计类似区块链：

- Commits 对应区块，实验对应工作量证明
- 验证便宜（训练一次就知道），但搜索昂贵（可能尝试一万个想法才有一个成功）
- 架构类似 SETI@Home、Folding@Home 等分布式计算项目

大胆推测

“互联网上的 Agent 集群有可能协作改进 LLM，甚至可能绕着前沿实验室跑圈。”

逻辑：前沿实验室有大量可信算力，但地球更大，有海量不可信算力。如果验证系统设计得好，分布式集群未必不能胜出。

进一步设想：关心某种癌症研究？可以购买算力加入该项目的 AutoResearch 池。算力可能成为贡献给公共事业的新货币。

3.5 本章小结

- AutoResearch 的核心理念是“把人从回路中移除”，让 AI Agent 自主循环实验
- 在 Karpathy 自认为已经调好的项目上仍获得了 11% 效率提升
- 研究组织可以被抽象为一组 Markdown 文件，进而实现元优化
- 分布式版本（Open Ground）可能让互联网 Agent 集群与前沿实验室竞争

4 AI 的参差性：天才 PhD 与 10 岁小孩

4.1 可验证领域 vs 不可验证领域

Karpathy 对 AI 自主循环的限制条件提出了两个关键观察。

AutoResearch 的适用边界

第一，这种方式极其适合有客观可衡量指标的任务（如优化 CUDA kernel：行为一致但速度更快）。但很多事情无法量化评估，那就无法做 AutoResearch。

第二，整个系统现在“在接缝处爆裂”。如果你试图走得太远，整体反而变成负价值。

Karpathy 用了一个极其精准的比喻：

参差性 (Jaggedness)

“我同时感觉在跟一个极其聪明的 PhD 系统程序员和一个 10 岁小孩对话。这太奇怪了，因为人类不会出现这种组合。”

人类的能力更加“耦合”——各方面水平差不多。但 Agent 有远超人类的参差性。有时你让它实现一个功能，回来的东西完全离谱。

4.2 强化学习的优化盲区

Karpathy 分析了参差性的根源：模型通过强化学习 (RL) 训练，所以它们能改进的只有可验证的东西——程序是否正确？单元测试是否通过？但更“软”的能力（理解意图、知道何时追问）不在 RL 优化范围内。

一个直观的例子：问最先进的 ChatGPT 讲个笑话，最常出现的还是“为什么科学家不信任原子？因为它们组成了一切”——这个笑话三四年前就是这个，现在还是这个。模型在 Agent 任务上能连续跑几个小时，但讲笑话和五年前一模一样。

流行假说的挑战

“在可验证领域（编程、数学）变强就会在所有领域变强”——Karpathy 认为这个假说没有成立，或者只在令人不满意程度上成立。有些领域在被优化，有些不在，而这一切都压缩在不透明的神经网络里。

推论：我们不会免费地在所有领域获得智能和能力。

4.3 一个模型还是一千个大脑

面对参差性问题，是否应该把单体模型拆成可以分别优化的专业化版本？

Karpathy 认为当前实验室在追求“模型单一培养” (monoculture) ——一个模型什么都要聪明。但他觉得应该有更多“物种分化” (speciation)：保留认知核心但特化到具体任务。

物种分化的瓶颈

通过 context window 做定制简单又便宜——这是当前个性化的主要方式。但真正触碰权重（微调、持续学习、领域特化）的技术还不成熟。动了权重就是在改变整个模型的智能，风险比改 context window 大得多。物种分化目前被技术瓶颈卡住了。

4.4 本章小结

- AI 的参差性 (jaggedness) 是当前 Agent 系统最突出的特征——超级智能与低级错误并存
- 根源在于 RL 只优化可验证领域，不可验证的“软能力”几乎停滞
- “可验证领域变强 = 所有领域变强”的假说并不成立
- 模型的物种分化是潜在出路，但被权重操纵技术的不成熟所制约

5 开源与闭源：中心化的历史记录很差

5.1 差距在缩小

Karpathy 观察到开源与闭源前沿的差距在持续收敛：从最初完全没有开源，到落后 18 个月，到现在约落后 6-8 个月。

他用 Linux 做类比——Linux 运行在绝大多数服务器上，因为行业需要一个共同的开放平台。AI 领域也有同样的需求。

开源面临的两个挑战

1. **资本支出是硬约束**——训练前沿模型需要大量资金，这让开源竞争更难
2. **前沿智能的需求依然存在**——诺贝尔奖级别的工作、“把 Linux 从 C 改写成 Rust”这种巨型项目，可能是闭源实验室的地盘

开源会蚕食更基础的用例。

5.2 为什么不回前沿实验室

Karpathy 选择保持独立，给出了几个原因：

1. **利益冲突**：在前沿实验室有巨大的财务激励。你一边造这项技术、一边从中获益、一边被财务手段深度绑定——这个矛盾至今没有真正解决。
2. **表达自由**：“在前沿实验室内部，有些话你不能说，有些话组织希望你说。不会扭你的胳膊，但你能感受到那种压力。”
3. **影响力的幻觉**：即使你参与决策讨论，当真正的高风险时刻到来时，作为员工你对组织行为的实际影响力有多大？

中心化的隐忧

Karpathy 对中心化天然警惕，引用了东欧的历史教训——“中心化的历史记录很差”。在机器学习中，集成模型永远优于单一模型；类比到决策，他希望“有更多人在房间里，当最难的决策到来时”。

更让他不安的是：即使在闭源端，领先者的圈子也在进一步缩小。

5.3 本章小结

- 开源与闭源的差距从 18 个月缩小到 6-8 个月
- Karpathy 选择独立以避免利益冲突和表达约束
- 他对 AI 领域权力中心化的趋势高度警惕
- 当前格局“意外地还不错”——闭源推进前沿，开源覆盖实用场景

6 就业市场与物理世界：原子比特难一百万倍

6.1 数字 AI 的三阶段路线图

Karpathy 提出了一个清晰的分析框架：

AI 渗透的三个阶段

1. **数字空间大规模解除束缚**——过去因为人类思考周期不够而没被充分处理的数字信息，现在被 AI 大量重写
2. **数字与物理的接口**——传感器（看见世界）和执行器（对世界做点什么）。很多有意思的公司会出现在这个接口上
3. **物理世界的全面渗透**——市场规模可能远大于数字空间，但难度也大一百万倍

当前正在发展的主要是“数字 AI”——可以在数字世界中操纵信息的实体，翻转比特比加速物质快一百万倍。

6.2 就业影响：杰文斯悖论

杰文斯悖论 (Jevons Paradox)

19 世纪经济学家提出：当技术进步提高资源使用效率时，资源消耗反而可能增加而非减少。经典案例是 ATM 与银行柜员——ATM 降低了网点运营成本，开了更多网点，反而雇了更多柜员。

注：美国银行柜员数量在 2010 年代确实开始下降，这个案例有其局限性。

Karpathy 认为类似的事情正在软件领域发生：软件变便宜了，海量被压抑的需求会释放。代码现在是“临时性的”、可修改的，整个数字基础设施有巨大的重写需求。

但他也坦承一个更尖锐的现实：

自动化的终极悖论

“我们在 OpenAI 的时候，我跟同事说，你们知道吗，如果我们成功了，我们所有人都失业了。我们本质上就是在给 Sam 或者董事会造自动化系统。”

自动化真的在起作用，连前沿实验室的研究人员也在经历焦虑。

6.3 机器人与信息市场

从 Tesla 自动驾驶经验出发，Karpathy 指出：十年前大量自动驾驶创业公司，大多数最终没活下来。原因是资本密集、时间漫长——原子实在太难了。

他还提出了一个有意思的缺失场景：信息市场。如果预测市场（如 Polymarket）有越来越多自主 Agent 参与，“从德黑兰拍一张照片应该值 10 美元”——不是人在看，是 Agent 在试图判断市场走势。

Karpathy 引用了 Daniel Suarez 的科幻小说《Daemon》：书中的超级智能把人类既当传感器又当执行器，社会围绕机器的需求重新组织。他觉得某种类似的事情正在发生。

6.4 本章小结

- 数字空间会先被大规模改造，物理世界渗透难度高一百万倍
- 杰文斯悖论暗示软件工程师的需求可能不减反增，但长期走向存在不确定性
- AI 从数字到物理的扩展将经历三个阶段，数字/物理接口是近期机会所在
- 信息市场和 Agent 驱动的预测市场是尚未充分开发的方向

7 MicroGPT 与 Agent 时代的教育

7.1 200 行代码的极简 GPT

MicroGPT (2026 年 2 月发布) 是 Karpathy 十几年来反复简化 LLM 到本质的痴迷的终点。约 200 行纯 Python，零依赖，包含完整 GPT 训练和推理所需的一切：数据集、分词器、自动微分引擎、GPT-2 架构、Adam 优化器、训练循环、推理循环。

MicroGPT 的定位

Karpathy 将 MicroGPT 称为“艺术项目”。它的意义在于证明：训练神经网络所需的全部算法内容其实极其简洁，其余数百万行代码都是为了效率。这是 nanoGPT、micrograd、makemore 系列的延续和终点。

7.2 教育范式的转变：向 Agent 解释

Karpathy 发现了一个对教育有深远影响的变化。过去他会为 MicroGPT 录视频逐行讲解，但他意识到这已经不太需要了——代码就 200 行，任何人都可以让 Agent 用各种方式解释它。

教育的未来方向

“我不再是向人类解释了。我是向 Agent 解释。如果 Agent 理解了，它们可以做路由，用读者的语言、按读者的水平、以无限的耐心来讲解。”

教育正在被重定向：

- 不是为人写 HTML 文档，而是为 Agent 写 Markdown 文档
- 不是你向人解释，而是 Agent 替你解释
- 你的工作是提供 Agent 做不到的那几个关键洞察

一个关键验证：Karpathy 试过让 Agent 自己从零写出 MicroGPT 的极简版本——做不到。200 行是他十几年痴迷的结晶，Agent 无法从零创造，但完全能理解它并解释为什么这样设计。

人类价值的新定义

“这就是我的价值贡献——那几个 bit。其他一切，Agent 都能做。”

Agent 做不到的事，才是你的工作。Agent 能做到的事，它们很快就能做得比你更好。

7.3 本章小结

- MicroGPT 用 200 行纯 Python 证明了 GPT 训练的核心算法极其简洁
- 教育的目标受众正在从人类转向 Agent——“向 Agent 解释，让 Agent 替你教学”
- Agent 能完美理解和解释代码，但无法从零创造极简设计——后者是人类的独特价值
- 未来的“课程”可能是指导 Agent 如何教学的元信息（skill/课程脚本）

8 总结与延伸

整场访谈中，Karpathy 反复回到“AI 精神错乱”（AI psychosis）这个词。这不是负面描述——更像是一个发现了无限可能但时间有限的人对自己状态的精准诊断。当你觉得一切都是“技能问题”的时候，每一分钟没在探索就是一分钟的浪费。

8.1 核心论点回顾

1. 工作模式已经不可逆地改变——从“写代码”到“指挥 Agent”，多 Agent 并行是新常态
2. 递归自我改进已经在起作用——AutoResearch 在成熟项目上仍能发现 11% 的优化空间
3. AI 的参差性是结构性的——可验证领域飞速进步，不可验证领域几乎停滞
4. 开源正在缩小差距——但前沿智能可能始终是闭源实验室的地盘
5. 数字空间先行，物理世界滞后——原子比特难一百万倍
6. 教育的对象从人变成了 Agent——人的价值在于 Agent 做不到的那几个 bit

8.2 三个值得持续关注的信号

- AutoResearch 的分布式版本（Open Ground）能否真正运转——这将直接影响前沿研究的格局：分布式 Agent 集群是否真能与中心化实验室竞争？
- 模型的参差性是暂时的还是结构性的——如果是结构性的，自主 Agent 的可靠性天花板就是有限的；如果是暂时的，全自主系统将很快到来
- 前沿实验室进一步中心化的趋势——当决策权集中在越来越少的人手里，Karpathy 所说的“不能说的话”会变成什么？

8.3 拓展阅读

- AutoResearch 开源仓库：github.com/karpathy/autoresearch
- MicroGPT 项目：github.com/karpathy/microgpt
- OpenClaw 项目：github.com/pspdfkit-labs/openclaw
- Daniel Suarez, 《Daemon》——书中描述了一个 AI 系统将人类作为传感器和执行器的未来社会
- 完整访谈视频：youtube.com/watch?v=kWSVtQ7dziU